

IN-PLACE MERGE SORT

Of compromises and the elegance lie

Florian Sihler

September 21, 2026
SP, Ulm University

IN-PLACE MERGE SORT

Of compromises and the elegance lie

Florian Sihler



Officially supported by the Ping-Ping-Uh-Uh-Foundation for Emotional Peeps. Peep n' peace and wa... oh look, there's cake!

September 21, 2026
SP, Ulm University

2.1

Merge Sort itself

[4]: *ETK-Package, tikzpingus*
Sihler, 2021



Merge Sort itself

L

2.3

Merge Sort itself

```
def sort(list: List) → List:
```

```
def merge(a: List, b: List) → List:
```

[4]: *ET_X-Package, tikzpingus*
Sihler, 2021

2.4

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;
```

```
def merge(a: List, b: List) → List:
```

2.5

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);
```

```
def merge(a: List, b: List) → List:
```

2.6

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
```

2.7

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
```

[4]: *ETK-Package*, tikzpingus
Sihler, 2021

2.8

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;
```

[4]: *ETK-Package*, tikzpingus
Sihler, 2021

2.9

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    |
```

2.10

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged ← merged ++ x;  
  ret merged;
```

[4]: *ETK-Package*, tikzpingus
Sihler, 2021

2.11

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));
```

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
    a not empty ? a : b
```

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);
```

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);  
  ret merged;
```

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);  
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package*, tikzpingus
Sihler, 2021

2.16

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```



```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);  
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package, tikzpingus*
Sihler, 2021



Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```



```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package*, tikzpingus
Sihler, 2021

2.18

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```



```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);  
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package*, tikzpingus
Sihler, 2021



2.19

Merge Sort itself

```
def sort(list: List) → List:  
  if len list ≤ 1 : ret list;  
  left, right ← split(list);  
  ret merge(sort(left), sort(right));
```



```
def merge(a: List, b: List) → List:  
  merged ← empty List;  
  while a & b aren't empty :  
    x ← a[0] ≤ b[0] ? a : b;  
    merged.add(x.remove(0));  
  merged.add(a not empty ? a : b);  
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package*, tikzpingus
Sihler, 2021



Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```



```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *BTCP-Package*, tikzpingus
Sihler, 2021

Merge Sort itself

```

def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));

```



```

def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;

```

We will only look at the ascending case. An example for the hordeeeee...



[4]: *ETK-Package*, tikzpingus
Sihler, 2021

Merge Sort itself

```

def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));

```

```

def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));

  merged.add(a not empty ? a : b);
  ret merged;

```



We will only look at the ascending case. An example for the hordeeeee...



[4]: BTPK-Package, tikzpingus
Sihler, 2021

Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTpK-Package, tikzpingus
Sihler, 2021



Merge Sort itself

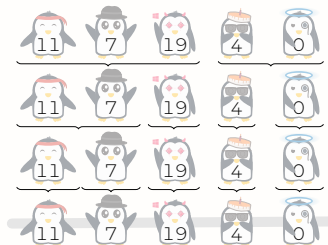
```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTPK-Package, tikzpingus
Sihler, 2021



Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));

  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTPK-Package, tikzpingus
Sihler, 2021



Merge Sort itself

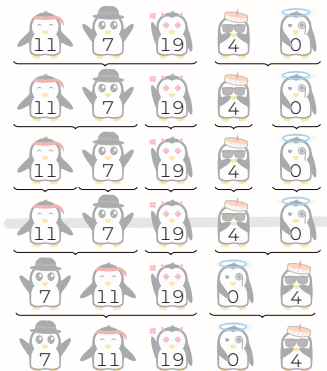
```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTpK-Package, tikzpingus
Sihler, 2021



Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTPK-Package, tikzpingus
Sihler, 2021



3.1

In-Place



3.2

In-Place

- The meaning varies:



3.3

In-Place

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.



3.4

In-Place

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations
 - Constant additional pointer/index accesses

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations
 - Constant additional pointer/index accesses
 - Constant memory to manipulate the input

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations
 - Constant additional pointer/index accesses
 - Constant memory to manipulate the input
 - ...

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations
 - Constant additional pointer/index accesses
 - Constant memory to manipulate the input
 - ...
- If not in-place: “out-of-place”

4.1

Nothing is free



4.2

Nothing is free

- We improve the runtime by:



4.3

Nothing is free

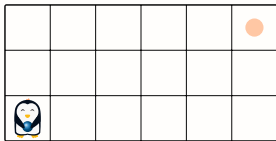
- We improve the runtime by:
 - removing redundant operations



4.4

Nothing is free

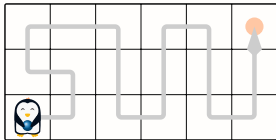
- We improve the runtime by:
 - removing redundant operations



4.5

Nothing is free

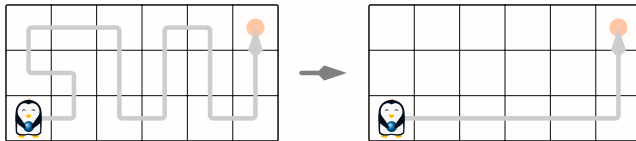
- We improve the runtime by:
 - removing redundant operations



4.6

Nothing is free

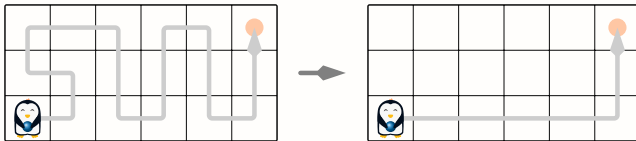
- We improve the runtime by:
 - removing redundant operations



4.7

Nothing is free

- We improve the runtime by:
 - removing redundant operations

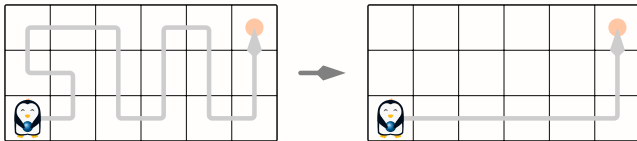


- exploiting the constraints (e.g. numbers only)

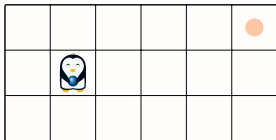
4.8

Nothing is free

- We improve the runtime by:
 - removing redundant operations



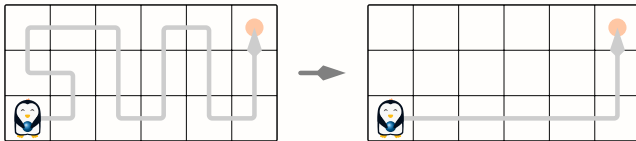
- exploiting the constraints (e.g. numbers only)



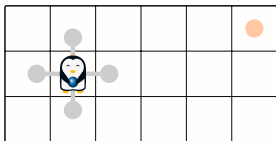
4.9

Nothing is free

- We improve the runtime by:
 - removing redundant operations



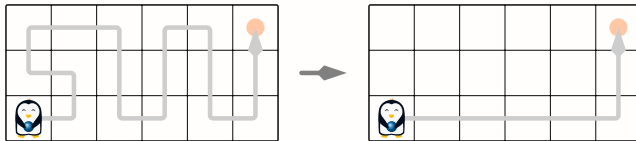
- exploiting the constraints (e.g. numbers only)



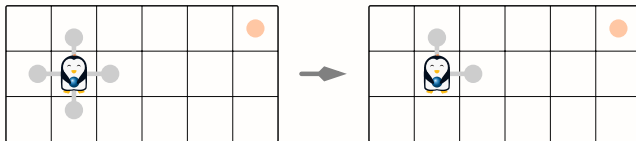
4.10

Nothing is free

- We improve the runtime by:
 - removing redundant operations



- exploiting the constraints (e.g. numbers only)



5.1

Nothing is free

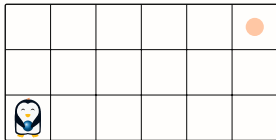


- We improve the runtime by:

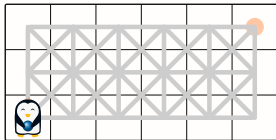


- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)

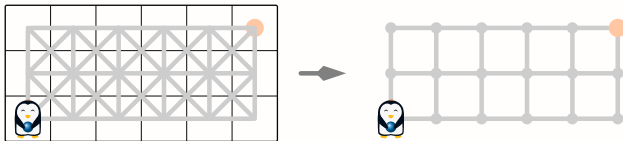
- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)



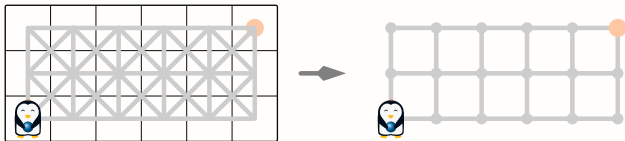
- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)



- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)



- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)



- additional memory (e.g. as a cache)

- We improve the runtime by:

- We improve the runtime by:
 - removing redundant operations
 - exploiting the constraints
 - abstracting cleverly
 - additional memory

- We improve the runtime by:
 - removing redundant operations
 - exploiting the constraints
 - abstracting cleverly
 - additional memory

- We improve the runtime by:
 - removing redundant operations
 - exploiting the constraints
 - abstracting cleverly
 - additional memory
- Most common compromise: memory vs. runtime

7.1

Pointer Comparison



InplaceMerge.java



Classics > **A Better Merge** > A Better Split > Conclusion



7.2

Pointer Comparison



```
void merge(int[] arr, int l, int m, int r) {
```

```
}
```



InplaceMerge.java



7.3

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;
```

```
}
```



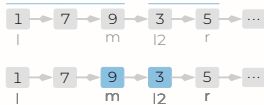
InplaceMerge.java



7.4

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
  
    }  
}
```



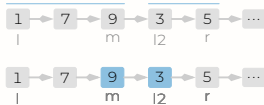
InplaceMerge.java



7.5

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
    while(l <= m && l2 <= r) {  
  
  
    } }  
}
```



InplaceMerge.java



Classics



A Better Merge



A Better Split



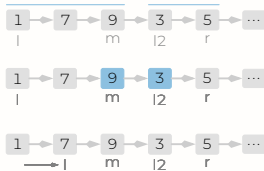
Conclusion



7.6

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
    while(l <= m && l2 <= r) {  
        if(arr[l] <= arr[l2]) { l++; }  
  
        }  
    }
```



InplaceMerge.java



Classics



A Better Merge



A Better Split



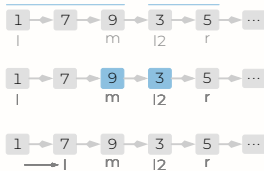
Conclusion



7.7

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
    while(l <= m && l2 <= r) {  
        if(arr[l] <= arr[l2]) { l++; }  
        else {  
  
        }  
    }  
}
```



InplaceMerge.java



Classics



A Better Merge



A Better Split



Conclusion



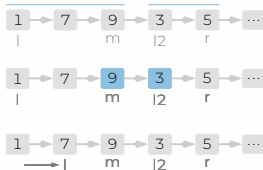
7.8

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right

```

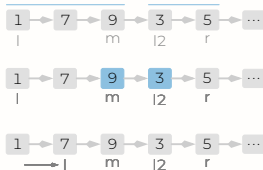
```
    } } }
```



7.9

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
    while(l <= m && l2 <= r) {  
        if(arr[l] <= arr[l2]) { l++; }  
        else { // shift elements from l, 1 to right  
            int shift = arr[l2], idx = l2;  
  
            }  
    }  
}
```



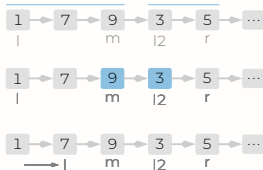
InplaceMerge.java



7.10

Pointer Comparison

```
void merge(int[] arr, int l, int m, int r) {  
    int l2 = m + 1;  
    if(arr[m] <= arr[l2]) return;  
    while(l <= m && l2 <= r) {  
        if(arr[l] <= arr[l2]) { l++; }  
        else { // shift elements from l, 1 to right  
            int shift = arr[l2], idx = l2;  
            while(l < idx) {  
  
            }  
  
        }  
    }  
}
```



InplaceMerge.java



Classics



A Better Merge



A Better Split



Conclusion



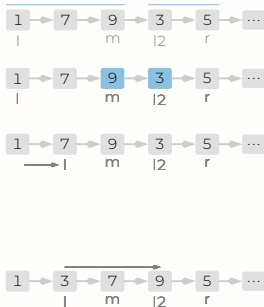
7.11

Pointer Comparison

```

void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right
            int shift = arr[l2], idx = l2;
            while(l < idx) {
                arr[idx] = arr[idx - 1]; idx--;
            }
        }
    }
}

```



InplaceMerge.java



Classics



A Better Merge



A Better Split



Conclusion



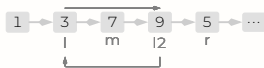
7.12

Pointer Comparison

```

void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right
            int shift = arr[l2], idx = l2;
            while(l < idx) {
                arr[idx] = arr[idx - 1]; idx--;
            }
            arr[l] = shift;
        }
    }
}

```



InplaceMerge.java



Classics



A Better Merge



A Better Split



Conclusion



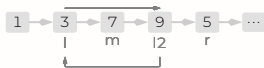
7.13

Pointer Comparison

```

void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right
            int shift = arr[l2], idx = l2;
            while(l < idx) {
                arr[idx] = arr[idx - 1]; idx--;
            }
            arr[l] = shift;
            l++; m++; l2++;
        }
    }
}

```



InplaceMerge.java



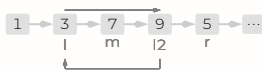
7.14

Pointer Comparison

Now the time complexity has gotten worse.
From $\mathcal{O}(n \log n)$ to $\mathcal{O}(n^2)$.



```
void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right
            int shift = arr[l2], idx = l2;
            while(l < idx) {
                arr[idx] = arr[idx - 1]; idx--;
            }
            arr[l] = shift;
            l++; m++; l2++;
        }
    }
}
```



InplaceMerge.java



8.1

Iterative Split

 IterativeSplit.java



Classics > A Better Merge > **A Better Split** > Conclusion



8.2

Iterative Split

```
void sort(int[] arr) {
    int n = arr.length - 1;
```

}

 IterativeSplit.java

8.3

Iterative Split

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
  
  
    }  
}
```

8.4

Iterative Split

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
  
            }  
        }  
    }  
}
```



IterativeSplit.java



Classics



A Better Merge



A Better Split



Conclusion



8.5

Iterative Split

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
        }  
    }  
}
```

8.6

Iterative Split

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
        }  
    }  
}
```

8.7

Iterative Split

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```

8.8

Iterative Split

And what happens
here? A wild lie...



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



IterativeSplit.java



Classics



A Better Merge



A Better Split



Conclusion





```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```

1. Each merge grows the list length faithfully by 2^i .

8.10

Iterative Split

And what happens here? A wild lie...



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```

1. Each merge grows the list length faithfully by 2^i .
2. We can start directly with single-element lists.

9.1

Iterative Split



9.2

Iterative Split



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



Classics



A Better Merge



A Better Split

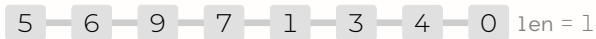


Conclusion



9.3

Iterative Split



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



9.4

Iterative Split



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



9.5

Iterative Split



The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



9.6

Iterative Split



The three small, light gray numbers mark the indices of `left`, `mid` and `right`.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



9.7

Iterative Split



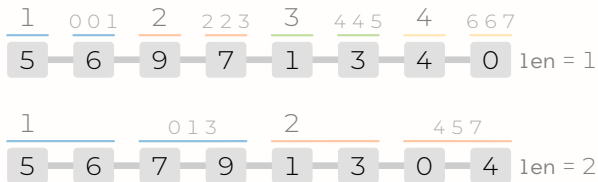
The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



9.8

Iterative Split



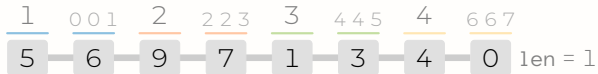
The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



9.9

Iterative Split



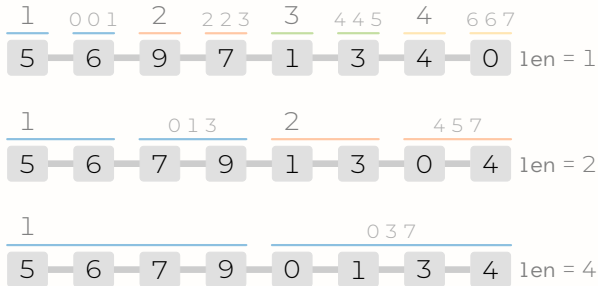
The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



9.10

Iterative Split



The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



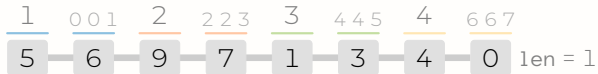
9.11

Iterative Split



The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



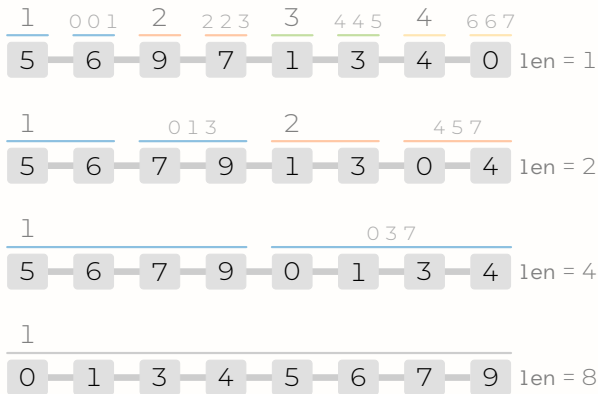
9.12

Iterative Split



The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



9.13

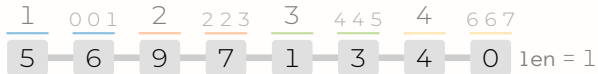
Iterative Split



In case the array would need only one merge, or none at all, to begin with.

The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



Classics



> A Better Merge

> **A Better Split**

> Conclusion



10.1

Iterative Split



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



Classics



A Better Merge



A Better Split



Conclusion



10.2

Iterative Split



What happens if the length
is not a power of two?

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



10.3

Iterative Split



What happens if the length
is not a power of two?

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



10.4

Iterative Split



What happens if the length
is not a power of two?

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



10.5

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```



10.6

Iterative Split



What happens if the length is not a power of two?



```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```

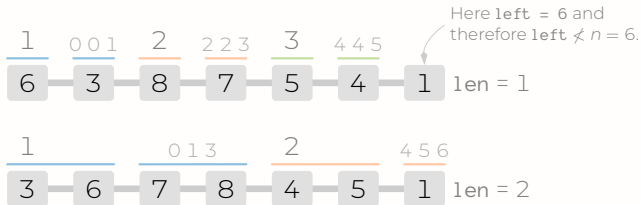


10.7

Iterative Split



What happens if the length is not a power of two?



```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```

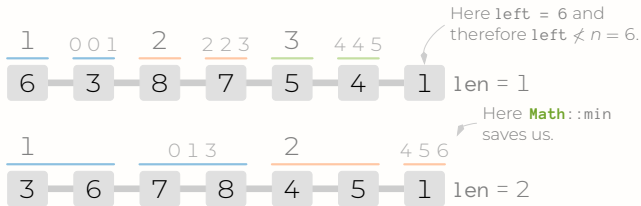


10.8

Iterative Split



What happens if the length is not a power of two?



```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```

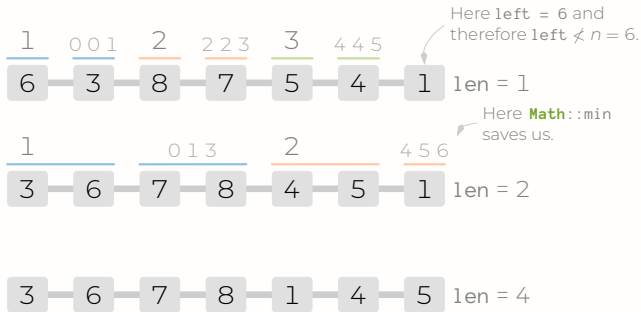


10.9

Iterative Split



What happens if the length is not a power of two?



```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



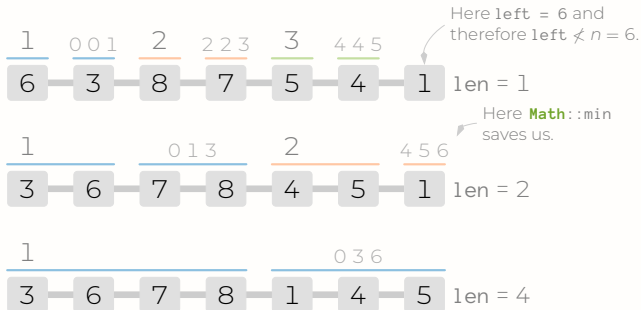
10.10

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



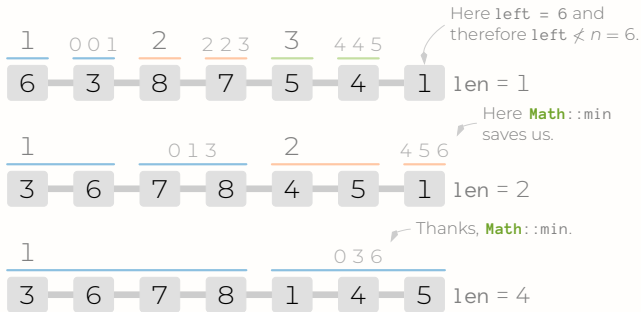
10.11

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



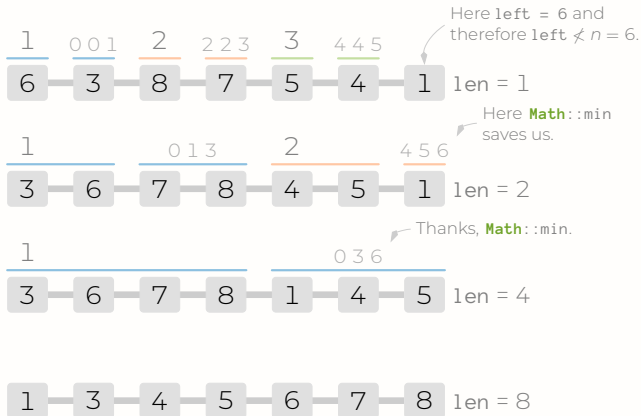
10.12

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



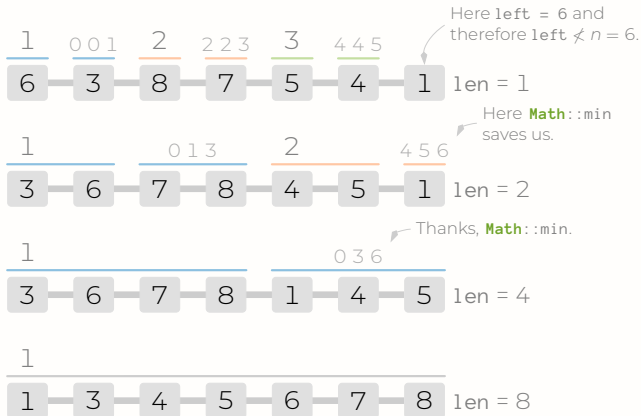
10.13

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



11.1

What remains to be noted...



11.2

What remains to be noted...

- The additional memory is now entirely in $\mathcal{O}(1)$

11.3

What remains to be noted...

- The additional memory is now entirely in $\mathcal{O}(1)$
- But the runtime is now in $\mathcal{O}(n^2)$

11.4

What remains to be noted...

- The additional memory is now entirely in $\mathcal{O}(1)$
- But the runtime is now in $\mathcal{O}(n^2)$
 - The iterative sort contributes “only” $\mathcal{O}(n \log n)$ to that

11.5

What remains to be noted...

- The additional memory is now entirely in $\mathcal{O}(1)$
- But the runtime is now in $\mathcal{O}(n^2)$
 - The iterative sort contributes “only” $\mathcal{O}(n \log n)$ to that
 - The complexity springs from the merge...

12.1

Closing Remarks

[2]: *Efficiently merging two sorted arrays with $O(1)$ extra space*, 2021

[5]: *The algorithm design manual*
Skiena, 2020

[3]: *Merge path-parallel merging made simple*
Odeh, 2012

[1]: *Sublinear merging and natural mergesort*
Carlsson, 1993



Classics



A Better Merge



A Better Split



Conclusion



12.2

Closing Remarks

- (Presumably) there is nothing below $\mathcal{O}(n \log^2(n))$

[2]: *Efficiently merging two sorted arrays with $\mathcal{O}(1)$ extra space*, 2021

[5]: *The algorithm design manual*
Skiena, 2020

[3]: *Merge path-parallel merging made simple*
Odeh, 2012

[1]: *Sublinear merging and natural mergesort*
Carlsson, 1993



12.3

Closing Remarks

- (Presumably) there is nothing below $\mathcal{O}(n \log^2(n))$
- That bound is reached with a Shell-Sort-inspired approach

[2]: *Efficiently merging two sorted arrays with $\mathcal{O}(1)$ extra space*, 2021

[5]: *The algorithm design manual*
Skiena, 2020

[3]: *Merge path-parallel merging made simple*
Odeh, 2012

[1]: *Sublinear merging and natural mergesort*
Carlsson, 1993



12.4

Closing Remarks

- (Presumably) there is nothing below $\mathcal{O}(n \log^2(n))$
- That bound is reached with a Shell-Sort-inspired approach
- There are many more derivatives, such as the “Natural Mergesort”...

[2]: *Efficiently merging two sorted arrays with $\mathcal{O}(1)$ extra space*, 2021

[5]: *The algorithm design manual*
Skiena, 2020

[3]: *Merge path-parallel merging made simple*
Odeh, 2012

[1]: *Sublinear merging and natural mergesort*
Carlsson, 1993



- [1] Svante Carlsson, Christos Levkopoulos, and Ola Petersson. “Sublinear merging and natural mergesort”. 1993
- [2] *Efficiently merging two sorted arrays with $O(1)$ extra space*. 2021
- [3] Saher Odeh et al. “Merge path-parallel merging made simple”. 2012
- [4] Florian Sihler. *L^AT_EX-Package, tikzpingus*. 2021
- [5] Steven S. Skiena. *The algorithm design manual*. 2020

Florian Sihler

Ulm, September 21, 2026

